

Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition

Saaheer Purav

saaheer.mitbl2024@learner.manipal.edu
Manipal Institute of Technology Bengaluru
Bengaluru, India

Supreet Halagali

supreet.mitbl2024@learner.manipal.edu
Manipal Institute of Technology Bengaluru
Bengaluru, India

Abstract

Large language models can generate short RTL fragments, but the gap between small-code generation and benchmark-level hardware synthesis remains substantial. ArchXBench exposes this gap sharply: its reported frontier-model results solve lower levels but collapse from Level 4 onward under direct prompting. This study asks whether the failure frontier is inherent to current models, or whether it can be crossed by verifier-grounded autonomous synthesis. Four conditions are compared: single-shot generation, monolithic golden-feedback repair, decomposition-guided iterative repair, and testbench-localized decomposition. On original ArchXBench contracts, the evaluated pipeline obtains clean solves across Levels 3–6, including robust Level-4 FFT/IFFT and floating-point pipeline solves, Level-5 convolution/image-processing designs, and Level-6 AES encryption/decryption with golden verification. This demonstrates that hard RTL rows reported as unsolved by direct LLM generation can become solvable when generation is embedded in an executable repair loop. At the same time, the results show that decomposition is not universally superior: monolithic golden-feedback repair is a strong baseline on several Level-5 and repaired-contract rows. The second contribution is an executable-contract audit that separates original benchmark solves from minimally repaired contract fixtures. The audit reveals both benchmark defects and genuine unsolved capability boundaries, showing that progress on autonomous RTL synthesis depends on trustworthy executable contracts as much as stronger generation methods.

CCS Concepts

• **Hardware** → **Hardware description languages and compilation**; **Electronic design automation**; • **Software and its engineering** → *Automatic programming*.

Keywords

RTL synthesis, Verilog generation, LLM agents, CEGIS, hardware verification, benchmark evaluation

ACM Reference Format:

Saaheer Purav and Supreet Halagali. 2027. Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition. In *Proceedings of 32nd Asia and South Pacific Design Automation Conference (ASP-DAC '27)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

Autonomous code agents are increasingly judged by whether they can close a complete executable loop: generate a candidate, run a

checker, inspect failures, and repair. For software tasks, this loop has already changed what counts as a meaningful benchmark result. For hardware design, the corresponding question is more demanding: can an autonomous agent synthesize a nontrivial RTL design that survives simulation, timing/protocol assumptions, and golden-output comparison?

This study argues that hard RTL synthesis is an important test of autonomous design automation. RTL behavior depends on bit widths, signedness, clocking, reset protocols, pipeline latency, streaming handshakes, and external file-output conventions. A design may compile while still being functionally wrong; a simulator may run to completion while still producing incorrect golden traces.

This work studies this problem on ArchXBench, a recent benchmark suite for complex RTL synthesis. Unlike VerilogEval [4] and RTLLM [5], which established automatic evaluation for Verilog generation on smaller designs, ArchXBench includes multi-level tasks spanning arithmetic kernels, FFTs, filters, image processing, matrix multiplication, and cryptography [8]. The ArchXBench paper reports that frontier models solve lower-level tasks but fail from Level 4 onward under zero-shot pass@5 evaluation. This reported Level-4 boundary is the starting point of this paper. If an autonomous synthesis loop can solve rows beyond that boundary, the result is not merely an incremental prompt improvement; it changes the evaluation question from “can an LLM emit the whole design in one shot?” to “can a verifier-grounded design agent converge to correct RTL?”

The central question addressed in this study is: *can verifier-grounded autonomous synthesis solve hard RTL benchmark tasks that direct generation does not reliably solve, while preserving a clear distinction between model failures and benchmark-contract failures?*

The answer is partially affirmative, with important caveats. Four conditions are evaluated: a single-shot baseline (C1), a monolithic golden-feedback repair baseline (C2g), a decomposition-guided iterative repair pipeline (C4i), and a testbench-localized decomposition pipeline (C4tl). C4i and C4tl solve several rows that direct prompting does not solve, including hard Level-4 and golden-verified Level-5/Level-6 tasks. At the same time, C2g is a strong baseline and matches or exceeds decomposition on several rows. The contribution is therefore not a claim that modular decomposition dominates monolithic repair. Rather, the study shows that verifier-grounded repair and decomposition can cross important parts of the reported hard-task frontier, and that benchmark contracts must be audited before successes or failures are interpreted.

This paper makes three contributions:

- (1) It demonstrates that verifier-grounded autonomous synthesis can solve hard ArchXBench rows beyond the direct-generation

ASP-DAC '27, Tokyo, Japan
2027.

failure frontier, including robust original-contract Level-4 solves and golden-verified Level-5/Level-6 solves.

- (2) It compares single-shot generation, monolithic golden-feedback repair, decomposition-guided repair, and localized decomposition under a common artifact discipline, showing where each strategy succeeds or fails.
- (3) It introduces an executable-contract audit for hard RTL benchmarks, separating original-contract results, minimally repaired-contract results, held rows, and genuine remaining failures.

2 Related Work

2.1 LLM-Based RTL Generation

VerilogEval introduced an HDLBits-style benchmark for Verilog generation and enabled automatic functional checking of generated RTL [4]. RTLLM similarly proposed an open benchmark for design RTL generation with large language models [5]. These benchmarks were essential for making RTL generation measurable, but many tasks are small instructional circuits or moderate fragments rather than deeply structured systems.

More recent work moves beyond direct prompting. AutoChip uses compiler and simulator feedback to improve HDL generation [10]. VerilogCoder introduces graph-based planning and AST-based waveform tracing for autonomous Verilog coding [1]. AutoVeriFix uses simulation discrepancies to correct LLM-generated Verilog [9]. Spec2RTL-Agent studies LLM-agent systems for complex specification-to-RTL generation [14]. VFlow searches over agentic workflows for Verilog generation using Monte Carlo tree search [13]. HDLxGraph uses HDL-specific graph retrieval to support repository-level HDL tasks [15]. Evolutionary methods such as REvolution, Evolve, and COEVO use inference-time search or co-evolutionary optimization for RTL correctness and, in some cases, power/performance/area optimization [2, 6, 7]. RTL-BenchMT studies a complementary problem: agent-assisted maintenance of RTL benchmarks, including flawed-case revision and overfitting detection [12]. FormalRTL integrates software references and formal equivalence checking into a multi-agent RTL synthesis flow [3], while RTL-BenchLS proposes a large formally verified RTL benchmark for reasoning and generation tasks [11].

This study differs in emphasis. It evaluates on ArchXBench Levels 3–6, including rows at and above the reported Level-4 failure frontier. It also separates original benchmark contracts from repaired executable contracts. This distinction is important because an RTL benchmark can fail because the generated design is wrong, but it can also fail because the released testbench, file format, or golden comparator is inconsistent. RTL-BenchMT validates that benchmark ambiguity is a first-order issue for RTL-generation evaluation; FormalRTL and RTL-BenchLS show the value of stronger executable or formal oracles. Here, benchmark-contract audit is used to support a hard-ArchXBench synthesis study rather than to propose a general benchmark-maintenance or formal-equivalence benchmark.

Table 1 summarizes the positioning. Prior work has made substantial progress on LLM-aided RTL generation, repair, workflow search, and retrieval. The present study is complementary: it focuses on hard ArchXBench rows and on the reliability of the executable contracts used to judge those rows.

2.2 Verifier-Grounded Synthesis

Counterexample-guided inductive synthesis (CEGIS) uses a verifier to reject incorrect candidates and guide the next synthesis attempt. LLM-based repair loops instantiate a related pattern: generate code, run tests, inspect errors, and repair. For RTL, however, verifier feedback is heterogeneous. Failures can arise from syntax, elaboration, simulation, timing, protocol mismatch, file-output mismatch, or incorrect golden references. The present pipeline treats simulator and golden-output feedback as first-class signals, but also audits the benchmark contract when the executable artifacts disagree.

3 Problem Setting

Each ArchXBench design provides a natural-language specification and executable assets such as Verilog testbenches, input files, output files, and comparison scripts. An autonomous synthesis condition receives the task and emits RTL. For self-checking designs, a run is successful only when the official testbench passes all checks. For file-output designs, simulator completion is not sufficient; the generated output must match the golden output.

Three evaluation classes are distinguished throughout the paper:

- **Original-contract results:** runs against the released benchmark as-is.
- **Repaired-contract results:** runs against a separate copied fixture in which infrastructure or executable-contract inconsistencies are minimally repaired.
- **Held or excluded rows:** tasks for which the released contract remains ambiguous enough that a positive solve would be misleading.

This separation is necessary for credible RTL evaluation. If a checker is wrong, passing it may not mean solving the intended hardware task. If a checker cannot compare outputs correctly, failing it may not mean the model failed.

Table 2 gives the contract taxonomy used during the audit. The taxonomy is intentionally operational: each category changes how a run is interpreted. For example, a self-checking testbench can directly define a solve, while a display-only checker cannot.

4 Method

4.1 Evaluated Conditions

Table 3 summarizes the operational differences among the four conditions. All conditions receive the same problem description, module interface, and released system testbench. The difference is how much executable feedback is allowed and whether the generated design is treated monolithically or through a generated module structure.

C1: direct generation. The model is called five independent times with the specification and testbench excerpt, and the best candidate under the executable checker is retained. There is no repair feedback. This condition measures whether the task is already solved by direct emission.

C2g: monolithic golden-feedback repair. C2g keeps the whole design as one RTL artifact. The model first studies the specification, interface, and testbench, then emits a complete top module. Each round compiles and simulates the candidate. Compile errors, native self-checking failures, or file-output golden

Table 1: Positioning relative to recent LLM-aided RTL and verification work. The key distinction is not another prompt template, but a hard-ArchXBench study that combines verifier-grounded synthesis with explicit original-contract versus repaired-contract accounting.

Work	Primary target	Feedback / search	Main benchmark style	Difference from this study
VerilogEval / RTLLM	RTL generation benchmarks	Testbench evaluation	HDLBits-style / moderate RTL	Establishes automatic evaluation, but mostly easier than ArchXBench L4–L6
AutoChip / AutoVeriFix	HDL generation and repair	Compiler, simulator, oracle feedback	Verilog repair/generation tasks	Iterative correction is related; benchmark-contract audit is not central
VerilogCoder / Spec2RTL-Agent	Autonomous RTL agents	Planning, tracing, multi-agent generation	VerilogEval / RTLLM-style tasks	Strong agent baselines; not focused on ArchXBench hard-level contract failures
VFlow	Workflow search	MCTS over agent workflows	Verilog generation tasks	Searches agent workflows; does not study executable-contract validity
REvolution / Evolve / COEVO	Evolutionary RTL search	Population search and PPA/correctness feedback	VerilogEval / RTLLM-style tasks	Search-oriented; this study emphasizes ArchXBench hard rows and benchmark contracts
RTL-BenchMT	Benchmark maintenance	Agent-assisted flaw revision and overfitting probes	VerilogEval / RTLLM / CVDP-style tasks	Validates benchmark-contract concerns; this study focuses on hard-design synthesis
This work	Hard RTL synthesis	Golden repair, decomposition, contract audit	ArchXBench L3–L6	Separates original solves, repaired-contract solves, and held rows

Table 2: Executable-contract taxonomy used to prevent false wins and false failures. This taxonomy is central to interpreting hard RTL benchmark results.

Contract type	Paper treatment
Self-checking testbench	Pass all official checks; native pass score is sufficient.
File-output with golden	Require post-simulation golden comparison; simulation completion is diagnostic only.
Display-only checker	Not a solve until expected values are converted into executable checks.
Stale comparator or schema mismatch	Repair only in a separate copied fixture and report separately.
Spec/testbench disagreement	Exclude or hold unless a principled source of truth is available.
Missing oracle	Hold; do not claim positive results.

Table 3: Operational differences among evaluated synthesis conditions.

Cond.	Decomp.	Ref. gate	Golden fb.	Local repair	Budget
C1	no	no	no	no	5 attempts
C2g	no	no	yes	no	30 rounds
C4i	yes	checked	yes	module order	≈28 module rounds
C4tl	yes	enforced	yes	trace-lifted	3 gates + ≈28 rounds

mismatches are appended to the same conversation, and the model must return a full corrected module. The loop stops when the self-checking testbench passes, when the golden comparator reports all outputs correct, or after 30 repair rounds. This is a deliberately strong baseline: it uses the same executable oracle as the decomposed methods without paying module-integration risk.

C4i: decomposition-guided iterative repair. C4i first asks the model to decompose the task into submodules, interfaces, a top-level composition, and reference implementations. The generated reference composition is checked against the system testbench and then used as scaffolding for module-wise synthesis. For each submodule, C4i studies the local interface, neighboring modules, the reference implementation, and the system testbench, then repairs that module while other unresolved modules are held at their references. The per-module repair budget is approximately $28/n$ rounds for n generated submodules. After all modules are processed, the complete composed design is evaluated by the original system checker or golden comparator.

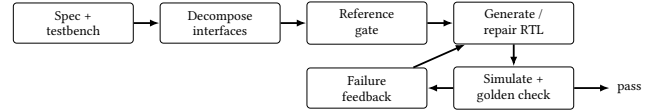


Figure 1: Verifier-grounded decomposition and repair loop. The pipeline converts one-shot RTL generation into an executable synthesis process: decompose, validate the reference composition, generate RTL, simulate, compare against golden behavior, and repair.

C4tl: testbench-localized decomposed repair. C4tl adds a stricter reference gate and a localization loop. It may retry decomposition up to three times, and it aborts if the generated top plus reference submodules cannot pass the system-level checker. After generating candidate implementations for all submodules, C4tl runs the full design. On failure, it performs trace-lifted localization: each candidate module is replaced by its validated reference while the rest of the candidate design is held fixed, and the module whose replacement most improves the system score is selected as the culprit. Only that module is repaired using the failing simulator output, golden mismatch details when available, the per-module swap scores, and the reference implementation. The loop repeats until the system checker passes or the call budget is exhausted.

4.2 Artifact and Verification Discipline

Each run is recorded under the repository artifact tree with a result file. Artifact-backed rows include generated RTL and, where applicable, decomposition metadata. File-output designs require strict golden verification; native simulator completion alone is treated as diagnostic evidence. Trusted score-only rows are labeled separately and are not described as artifact-backed.

5 Experimental Setup

5.1 Benchmark Scope

The evaluation covers ArchXBench Levels 3–6. Lower levels are not emphasized because the benchmark paper already reports that current models can solve lower-level tasks. The central interest is

the harder regime that includes iterative arithmetic, FFT/IFFT designs, floating-point pipelines, FIR filters, image-processing kernels, matrix multiplication, and AES.

5.2 Seeds, Model, and Metrics

Main paper tables use seeds 42, 123, and 456. Two additional C4tl Level-4 seeds, 789 and 1024, are used as robustness evidence. These are arbitrarily chosen fixed seeds used for reproducibility, not tuned seeds. The primary model label is gpt-5.5, as recorded in the repository artifacts.

For self-checking designs, a clean solve requires `best_passes == total_tests` with `total_tests > 0`. For file-output designs, a clean solve requires `golden_correct == golden_total` with `golden_total > 0`. Solve rates are reported over seeds.

5.3 Implementation Details

All methods are implemented as executable synthesis loops rather than manual prompt transcripts. The runner records the design, condition, model, seed, pass counts, golden counts, LLM-call count, generated RTL, and decomposition metadata when applicable. C1 uses five independent generations. C2g uses a 30-round monolithic repair loop. C4i allocates a fixed module-wise budget derived from a 28-round repair budget. C4tl uses at most three reference-decomposition attempts, then a bounded full-system test, localization, and repair loop.

All runs are evaluated through Icarus Verilog simulation. For self-checking rows, a solve requires all official checks to pass with a nonzero test count. For file-output rows, simulator completion is only diagnostic; a solve requires the generated output file to match the golden output exactly under the repository comparator. If a C4tl reference decomposition fails the original system checker, the run is scored as a failed decomposition rather than allowed to continue with an invalid scaffold.

6 Crossing the Original-Contract Frontier

Table 4 summarizes the main original-contract results. The strongest Level-3 method-value evidence is produced by C4i, which solves five selected Level-3 rows on all three seeds while C1 and C2g fail or nearly fail on those rows. The strongest hard-level coverage evidence is produced by C4tl on Level 4: the FFT/IFFT and floating-point pipeline rows solve on all three main seeds, and remain solved on two additional robustness seeds. At Levels 5 and 6, golden-verified solves are obtained on convolution/image-processing and AES rows.

The headline result is that original ArchXBench rows above the reported direct-generation failure frontier are no longer uniformly out of reach. The Level-4 FFT/IFFT rows are especially important because they are clean self-checking original-contract solves rather than repaired-contract results. The Level-6 AES rows further show that file-output/golden verification can be satisfied, not merely that a simulator runs to completion. Some rows at the boundary are not exclusive hard-frontier evidence. `fp_adder_pipeline`, `fp_mult_pipeline`, and `conv1d` are solved by C1 as well as feedback conditions; they are retained for coverage and calibration. The stronger frontier evidence comes from rows where C1 fails but

verifier-grounded conditions solve, such as the Level-4 FFT/IFFT rows and multiple Level-5/Level-6 golden-verified rows.

The results are deliberately not presented as a universal decomposition win. C2g is a serious baseline: it solves Level-4 FFT/IFFT rows, several Level-5 rows, and multiple repaired-contract rows. The most accurate interpretation is that decomposition provides value on specific hard rows and gives a structured synthesis process, while monolithic golden-feedback repair is often sufficient and sometimes better.

Case study: original Level-4 FFT. `fft_16pt_iterative` is a self-checking original-contract row, so no benchmark repair is involved. C1 fails on all three main seeds, while C2g and C4tl solve all three. The row therefore isolates the central mechanism: executable feedback can turn a direct-generation failure into a verified RTL solve without changing the released checker.

7 Executable-Contract Audit

Hard RTL benchmarks are useful only when their executable contracts reflect the intended task. During evaluation, several rows were found to contain inconsistent specifications, stale comparators, display-only checkers, hidden assumptions, or output-schema mismatches. The released benchmark is not overwritten. Instead, repaired fixtures are kept under a separate artifact root and are reported separately from original-contract results.

This audit is not a peripheral cleanup step. It is a prerequisite for credible autonomous RTL synthesis evaluation. At high difficulty levels, a benchmark row can fail because the model is incapable, because the generated RTL is wrong, because the output format is mismatched, or because the executable checker does not encode the intended task. Without separating these cases, a leaderboard can reward the wrong behavior or hide genuine progress.

The audit follows a fixed protocol. First, the released fixture is run unchanged. Second, the checker is classified as self-checking, file-output with golden reference, display-only, stale/schema-mismatched, specification-inconsistent, or missing-oracle. Third, file-output rows are judged only by post-simulation golden comparison. Fourth, infrastructure repairs are made only in copied fixtures and only when the intended oracle is recoverable from the released assets or an independently validated reference. Finally, unresolved rows are held rather than converted into positive results.

Table 5 reports the repaired-contract evidence. These rows are not counted as original ArchXBench solves. They answer a narrower question: when an infrastructure or executable-contract inconsistency is minimally repaired and oracle-validated, do the synthesis agents solve the intended task?

Why this is not test hacking. Original-contract rows are evaluated on the released benchmark without modification. Repaired-contract rows are stored under separate artifact roots, labeled separately in every table, and excluded from original ArchXBench solve claims. Repairs target executable infrastructure such as stale comparators, file schemas, or display-only checks; rows without a defensible oracle are held. The negative `systolic_gemm` result is important: even after making its checker executable, all methods remain at 0/3.

The audit produces both positive and negative evidence, which is why it is not simply a mechanism for producing extra wins. For

Table 4: Original-contract results on ArchXBench. Values are clean solves over seeds; bold 3/3 highlights full success on the main seed set. GV denotes golden-verified file-output rows. “+2 robust” means seeds 789 and 1024 also solve, giving 5/5 total for that row.

Level	Design	C1	C2g	C4i	C4tl	Evidence / interpretation
L3	fp_adder	0/5	1/5	3/3	3/3	C4i/C4tl improve over direct generation
L3	fp_multiplier	0/5	0/3	3/3	3/3	clean decomposition-guided solve
L3	gauss_siedel	0/3	0/3	3/3	1/3	C4i stronger than C4tl
L3	gradient_descent	0/3	0/3	3/3	3/3	clean decomposition-guided solve
L3	newton_raphson_sqrt	0/3	0/3	3/3	3/3	clean decomposition-guided solve
L3	newton_raphson_polynomial	0/3	0/3	0/3	0/3	original checker has unsatisfiable residual assertions; repaired C2g solves 3/3
L4	fft_16pt_iterative	0/3	3/3	0/3	3/3 (+2 robust)	hard-row solve; robustness seeds for C4tl
L4	ifft_16pt_iterative	0/3	3/3	0/3	3/3 (+2 robust)	hard-row solve; robustness seeds for C4tl
L4	fp_adder_pipeline	3/3	3/3	3/3	3/3 (+2 robust)	all methods solve
L4	fp_mult_pipeline	3/3	3/3	3/3	3/3 (+2 robust)	all methods solve
L5	conv1d	3/3	3/3	3/3	1/3	GV; C4i clean
L5	conv2d	0/3	3/3	0/3	0/3	GV; monolithic repair strongest
L5	dct_idct_8pt_pipelined	0/3	3/3	0/3	0/3	GV; monolithic repair strongest
L5	harris_corner_detection	0/3	3/3	3/3	0/3	GV; C4i matches C2g
L5	unsharp_mask	0/3	3/3	0/3	0/3	GV; artifact-backed C2g
L6	aes_encryption	3/3	3/3	3/3	2/3	GV; all major conditions strong
L6	aes_decryption	1/3	3/3	3/3	1/3	GV; repair improves direct generation

Table 5: Repaired-contract results. These rows are separate from original ArchXBench results and are used to distinguish benchmark-contract failures from model capability failures.

Design	C2g	C4i	C4tl
conv_3d	3/3	2/3	0/3
multich_conv2d	3/3	3/3	3/3
quantized_matmul	3/3	3/3	0/3
fp_band_pass_fir	3/3	0/1	0/1
fp_high_pass_fir	3/3	0/1	0/1
newton_raphson_polynomial	3/3	1/3	1/3
systolic_gemm	0/3	0/3	0/3

multich_conv2d and quantized_matmul, repairing the executable contract makes the intended task measurable and solvable. For systolic_gemm, converting display-only expected matrices into executable checks does not produce a win; all methods still fail. This is an important negative result because it shows that the audit is not merely a mechanism for converting failures into successes.

Case study: display-only GEMM checking. The released systolic_gemm checker prints expected matrices but does not reliably convert them into machine-readable pass/fail decisions. The repaired-contract fixture uses the expected matrices already present in the testbench to define executable comparisons. After this repair, C2g, C4i, and C4tl all remain at 0/3. This row is therefore reported as a genuine remaining capability boundary, not as a benchmark-repair success.

Several rows remain held or excluded. The L4 FIR family is excluded from positive tables because the specification and executable testbench disagree on filter coefficients/source-of-truth behavior. The L6 fp_low_pass_fir row is held because the released files do not expose an explicit coefficient/cutoff oracle. The L6 fft_streaming_64pt row is excluded because the released input/output contract has unresolved schema and numeric-encoding ambiguities.

8 Discussion

8.1 When Decomposition Helps

The results suggest a more nuanced conclusion than “decomposition is better.” Decomposition is most useful when a design naturally factors into modules and when failure localization reduces the repair burden. The Level-4 FFT/IFFT and floating-point pipeline rows support this conclusion: C4tl robustly solves all four core Level-4 rows on the main seeds and two additional robustness seeds. C4i also provides strong Level-3 evidence on numerical kernels where C1 and C2g fail. Notably, C4i fails on the FFT/IFFT rows where C4tl succeeds. One plausible explanation is that C4i uses a more aggressive decomposition strategy: for tightly coupled pipeline stages, finer-grained module boundaries can introduce integration risk that C4tl’s testbench-localized repair avoids.

For C4tl, the reference-composition gate is essential. A decomposition is not accepted merely because it appears plausible: the composed references must pass the system-level checker before trace-lifted localization begins. This requirement prevents arbitrary module boundaries from becoming a source of false confidence.

8.2 When Decomposition Does Not Help

C4i and C4tl do not dominate C2g. Several Level-5 and repaired-contract rows are solved most cleanly by monolithic golden-feedback repair. This can happen when the design is compact enough for whole-design repair, when module boundaries introduce integration risk, or when the primary difficulty is a global convention such as file format, fixed-point encoding, or streaming protocol. Therefore, the paper’s claim is intentionally conservative and, in our view, more useful for design automation: decomposition is a powerful synthesis strategy for some hard rows, but monolithic golden-feedback repair remains a strong tool that should be treated as a primary baseline.

8.3 Why Benchmark Contracts Matter

The benchmark audit is a design-automation contribution rather than an incidental engineering detail. LLM-aided RTL synthesis papers increasingly rely on executable benchmarks. If the executable contract is ambiguous, a model can fail for the wrong reason or pass the wrong task. The observed issues include hidden testbench assumptions, stale comparators, file-output schema mismatches, display-only checkers, unsatisfiable residual checks, and missing oracle information. Reporting these separately improves the reliability of future comparisons on hard RTL benchmarks.

9 Limitations

First, the evaluated methods do not establish universal dominance over monolithic repair. C2g is strong and should be treated as a primary baseline. Second, some historical rows are trusted score-only rather than artifact-backed; these rows are labeled accordingly in the repository and should not be described as artifact-backed until generated RTL is present. Third, repaired-contract evaluation depends on careful judgment about what constitutes an infrastructure repair rather than a semantic benchmark change. This risk is mitigated by keeping repaired fixtures separate, oracle-validating repairs when possible, and excluding unresolved rows. Fourth, the empirical study is concentrated on RTL synthesis. The broader lesson may transfer to other executable synthesis domains, but that transfer is not established here.

10 Conclusion

This study evaluated autonomous RTL synthesis on hard ArchXBench designs using iterative repair, modular decomposition, and strict simulator/golden feedback. The results cross a reported hard frontier: original-contract solves are obtained across Levels 3–6, including robust Level-4 rows and golden-verified Level-5/Level-6 designs. The study also shows why this frontier should be interpreted carefully. Monolithic golden-feedback repair is a strong baseline, decomposition is powerful but not universally superior, and benchmark-contract defects can otherwise obscure both progress and failure.

The broader implication is that the next stage of RTL-generation research should not be judged only by one-shot code emission. Hard autonomous RTL synthesis requires structured generation, executable feedback, and trustworthy benchmark contracts together. Without all three, both successes and failures can be misinterpreted; with all three, benchmark-level hardware synthesis begins to move from direct generation toward autonomous design automation.

References

- [1] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [2] Wei-Po Hsin, Ren-Hao Deng, Yao-Ting Hsieh, En-Ming Huang, and Shih-Hao Hung. 2026. Evolve: Evolutionary Search for LLM-based Verilog Generation and Optimization. *arXiv preprint arXiv:2601.18067* (2026).
- [3] Kezhi Li, Min Li, Xiangyu Wen, Shibo Zhao, Jieying Wu, Junhua Huang, and Qiang Xu. 2026. FormalRTL: Verified RTL Synthesis at Scale. *arXiv preprint arXiv:2603.08738* (2026).
- [4] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*.

- [5] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).
- [6] Kyungjun Min, Kyumin Cho, Junhwan Jang, and Seokhyeong Kang. 2026. REvolution: An Evolutionary Framework for RTL Generation Driven by Large Language Models. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [7] Heng Ping, Peiyu Zhang, Shixuan Li, Wei Yang, Anzhe Cheng, Shukai Duan, Xiaole Zhang, and Paul Bogdan. 2026. COEVO: Co-Evolutionary Framework for Joint Functional Correctness and PPA Optimization in LLM-Based RTL Generation. *arXiv preprint arXiv:2604.15001* (2026).
- [8] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A Complex Digital Systems Benchmark Suite for LLM Driven RTL Synthesis. *arXiv preprint arXiv:2508.06047* (2025).
- [9] Yan Tan, Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [10] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. In *Proceedings of the ACM/IEEE Design Automation Conference*.
- [11] Jing Wang, Shang Liu, Wenji Fang, Yuchao Wu, Yugao Zhu, and Zhiyao Xie. 2026. RTL-BenchLS: A Large-Scale Benchmark for RTL Reasoning and Generation with Large Language Models. *arXiv preprint arXiv:2606.08976* (2026).
- [12] Jing Wang, Shang Liu, Hangan Zhou, and Zhiyao Xie. 2026. RTL-BenchMT: Dynamic Maintenance of RTL Generation Benchmark Through Agent-Assisted Analysis and Revision. In *Proceedings of the ACM/IEEE Design Automation Conference*. doi:10.1145/3770743.3804053
- [13] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2026. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [14] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine Lin, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. *arXiv preprint arXiv:2506.13905* (2025).
- [15] Pingqing Zheng, Jiayin Qin, Fuqi Zhang, Shang Wu, Yu Cao, Caiwen Ding, and Yang Zhao. 2025. HDLxGraph: Bridging Large Language Models and HDL Repositories via HDL Graph Databases. *arXiv preprint arXiv:2505.15701* (2025).